

CWC 2.0 ESM — Request Lifecycle & Workflow-Designer Engine

Source of truth: backend/src/routes/request.routes.ts, backend/src/controllers/request.controller.ts, backend/src/services/requestTransition.service.ts, backend/src/utils/workflowTransitions.ts, backend/src/services/availableTransitions.service.ts, and the workflow-engine services (workflowCompiler, workflowGraph, workflowVersion, workflowValidator, workflowCommand, transitionGuards, transitionPolicy).

1. The most important concept: TWO transition systems

There are **two** sources of truth for request transitions. Understanding which is authoritative at runtime is essential:

- 1 **Published workflow-designer engine (AUTHORITATIVE at runtime).** Workflows are authored as versioned graphs (WorkflowType → WorkflowVersion → WorkflowNode + WorkflowEdge), validated, and **compiled** into runtime artifacts: the workflowTransition DB table and workflowStep rows. At runtime, requestTransition.service.ts and availableTransitions.service.ts read **only the DB table**. Published DB rows drive behavior.
- 2 **Hardcoded VALID_TRANSITIONS map** (backend/src/utils/workflowTransitions.ts) — **seed/documentation fallback ONLY**. isValidTransition (line 129): if a workflow_transitions row exists for the pair → true. If the table has **any** seeded rows but no active pair → **false** (never re-permits a deactivated row). Falls back to the map **only when the table is entirely empty** (unseeded env). Editing VALID_TRANSITIONS alone does **not** change runtime behavior.

WorkflowFormSchema / WorkflowFormResponse **do NOT exist as Prisma models**. Forms are stored as Json (RequestType.formConfig, Request.formConfigSnapshot) and validated by conditionalRules.service.ts / conditionEvaluator.service.ts.

2. Request creation (request.controller.ts:614 createRequest)

Pipeline:

- 1 **Resolve creation policy** via resolveRequestCreationPolicy (line 629) — server-resolved request type / service desk / tenant / SLA / confidentiality.
- 2 **Sanitize** summary/description (rich text allowed only for IT desk).
- 3 **Reference number** generated atomically via generateRequestRefNum (line 647, RequestCounter counter table, P2-11 race fix).
- 4 **Initial status hardcoded per request-type code** (lines 710–722): generic SUBMITTED; onboarding ONBOARDING_SUBMITTED; offboarding OFFBOARDING_SUBMITTED; purchase requisition/budget PENDING_CEO_APPROVAL_FIN; chargeback SUBMITTED; expense claim PENDING_MANAGER_APPROVAL_FIN; ESM travel PENDING_CEO_APPROVAL.
- 5 **SLA due date** computed from creationPolicy.slaHours.
- 6 In a transaction: prisma.request.create with status: initialStatus, optional assignedToId, slaDueAt, formConfigSnapshot + formConfigVersion; then creates structured records (ITHardwareRequest, OnboardingRequest + seed tasks, OffboardingRequest), initial RequestActivity, and RequestApproval PENDING rows for travel CEO / finance CEO.
- 7 Post-commit: applyEntityRouting, autoAssignRequest (+ assignment activity + REQUEST_ASSIGNED notification), requester REQUEST_CREATED notification.

3. Request routes (`request.routes.ts`, 160 lines)

Method	Path	Handler	Permission
GET	<code>`/requests`</code>	<code>`getAllRequests`</code>	auth + visibility scoping
POST	<code>`/requests`</code>	<code>`createRequest`</code>	auth
GET	<code>`/requests/pending-approvals`</code>	<code>`getPendingApprovals`</code>	<code>`request:approve`</code>
POST	<code>`/requests/bulk-action`</code>	<code>`bulkAction`</code>	<code>`request:approve`</code>
GET	<code>`/:id/available-transitions`</code>	<code>`getAvailableTransitions`</code>	authorizeResource
GET	<code>`/requests/recent-services`</code>	<code>`recentServices`</code>	auth
POST	<code>`/requests/export/xlsx`</code>	<code>`exportRequestsXlsx`</code>	<code>`request:export`</code>
GET	<code>`/:id`</code>	<code>`getRequestById`</code>	authorizeResource + access policy
PUT	<code>`/:id`</code>	<code>`updateRequest`</code>	authorizeResource
DELETE	<code>`/:id`</code>	<code>`deleteRequest`</code>	<code>`request:delete`</code>
GET	<code>`/:id/export/pdf`</code>	<code>`exportRequestPdf`</code>	authorizeResource
GET/POST	<code>`/:id/activities`</code>	get/add activity	authorizeResource
POST/GET/DELETE	<code>`/:id/attachments`</code>	upload/download/delete	authorizeResource
PUT	<code>`/:id/assign`</code>	<code>`assignRequest`</code>	<code>`request:assign`</code>
PUT	<code>`/:id/status`</code>	<code>`updateStatus`</code>	<code>`request:update`</code>
USE	<code>`/:id/participants`</code>	participant routes	authorizeResource

Many routes use `authorizeResource(loadRequestScopeFromParam('id'), ...)` as P02-09 defense-in-depth row-level access, plus `requestAccess.service.ts` for full policy evaluation.

4. The central transition service — `requestTransition.service.ts` (464 lines)

Every status change goes through `transitionRequest()` (line 213). This centralizes validation, terminal timestamps, SLA pause/resume, auto-assignment, activity + audit logging, notifications, and guard conditions. **Do not scatter** `prisma.request.update({ status })` **across controllers**.

Pipeline:

- 1 Fetch request + tenant assert.
- 2 **Validate** via `isValidTransition` (unless `skipValidation`).
- 3 **Actor policy** — `canActorTransition` when actor set and not `skipTransitionPolicy`.
- 4 **Run guards** — `runGuards` over `from→to` + `*→to` keys.
- 5 `requiresComment` from transition metadata.
- 6 **Terminal timestamps** — uses `getStatusDefinitionForRuntime lifecycleType` when the catalog is available, else local `RESOLVE/CLOSE/COMPLETE_STATUSES` sets; sets `resolvedAt/closedAt/completedAt`.
- 7 **Auto-assignment** from `autoAssignUserId / autoAssignRole`.
- 8 **SLA clock mutation** (PAUSE/RESUME) derived from `workflowStep.slaPause`.
- 9 **Atomic command** — `executeWorkflowCommand` commits status/version/history/activity/audit/outbox as one unit.

10 **Notifications** — durable via outbox handler REQUEST_STATUS_CHANGED; skipNotifications marks outbox PUBLISHED.

Pluggable guards: registerTransitionGuard(key, fn) (line 138) — registry of from→to | *→to → GuardFn[]; returns null (allow) or message (reject).

5. Guard conditions ([transitionGuards.ts](#), 553 lines)

Side-effect-free pre-transition predicates registered on the transition registry (module-load side effects). Categories:

- 1 **Terminal guard** — belt-and-suspenders for skipValidation.
- 2 **Comment required for rejections** — wildcard *→REJECTED_*.
- 3 **IT assignment guard** — only assigned agent/admin advance IT procurement.
- 4 **IT / Finance / Shared desk guards** — *→IT_ONLY_TARGETS require desk IT; *→FINANCE_ONLY_TARGETS require FINANCE; *→SHARED_APPROVAL_TARGETS allow FINANCE OR ESM.
- 5 **Role-based approval guards** — CEO/CTO/CFO/Group DCEO/Manager/Finance Head/Hiring Manager for specific from→to pairs.
- 6 **LOA preconditions** — LOA_APPROVED→LOA_ISSUED needs approvedBy; LOA_ISSUED→LOA_ACCEPTED needs signed file; LOA_ACCEPTED→COMPLETED needs signed file.
- 7 **Onboarding completion** — all tasks COMPLETED before ONBOARDING_COMPLETED.
- 8 **Offboarding phase guards** — resignation letter + exit interview before FINAL_WEEK; EXIT_PROCEDURES + all tasks before OFFBOARDING_COMPLETED.

6. Actor policy — [transitionPolicy.service.ts](#) (109 lines)

Data-driven authorization replacing per-controller hasRole/executive lookups. canActorTransition (line 49): resolves most-specific workflow_transitions row; **no row** → **deny**. If both allow-lists empty → legacy fallback (LEGACY_EXECUTIVE_APPROVAL_ROLES for approval targets, else AGENT/ADMIN). Explicit allowedRoles / allowedExecutiveRoles authoritative otherwise.

7. Transactional command boundary — [workflowCommand.service.ts](#) (425 lines)

Every status change passes through executeWorkflowCommand (line 419), a \$transaction(ReadCommitted). executeWorkflowCommandInTransaction (line 143):

- Guards protected patch fields; computes commandFingerprint (SHA-256, ignoring runtime timestamps/audit).
- **Idempotency:** tenant-scoped idempotencyKey on WorkflowCommandResult; replay returns stored result.
- **Optimistic concurrency:** updateMany guarded by status + version; non-1 count → 409 status/version conflict.
- Writes atomically: request status + version{increment:1} + patch, WorkflowHistory row, RequestActivity STATUS_CHANGE (+ optional SLA activity), AuditLog STATUS_TRANSITION, OutboxEvent REQUEST_STATUS_CHANGED, optional transactionMutations.

8. Available transitions — [availableTransitions.service.ts](#) (107 lines)

getAvailableTransitionsForRequest (line 19): queries active workflow_transitions rows scoped to the request (most-specific-first). **Key logic (63–70)**: if a published workflow has scoped rows (workflowTypeId === request's), it is treated as a **complete source** and global rows are filtered out (prevents HR/Finance routes leaking into IT_SIMPLE); otherwise falls back to global rows. De-dupes per toStatus by specificity, filters by canActorTransition, and normalizes requiresComment=true for REJECTED/CANCELLED.

9. Workflow-designer engine

Authoring & version lifecycle

- **Authoring source**: WorkflowType → WorkflowVersion (DRAFT/ACTIVE/ARCHIVED) → WorkflowNode (STATUS) + WorkflowEdge.
- **Compiled artifacts the runtime enforces**: WorkflowTransition (transition rules) + WorkflowStep (stepper UI + SLA pause).
- **Version lifecycle** (workflowVersion.service.ts): createDraft (24) clones the active version's graph with fresh node IDs; publishVersion (116) validates, applies statusRemap to move stranded requests, archives the prior ACTIVE, activates the draft, and compiles. rollbackToVersion (160), discardDraft (186). One ACTIVE per type.

Compilation (workflowCompiler.service.ts)

projectGraph (80) converts nodes→ProjectedStep, edges→ProjectedTransition (only status→status edges; BPMN-lite node types not yet supported). compileVersionInTransaction (179) does **delete-then-insert scoped to one** workflowTypeId (never touches workflowTypeId: NULL platform defaults). reverseCompile (207) + diffProjection (305) support shadow-mode comparison/backfill of legacy workflows into version 1.

Validation (workflowValidator.service.ts)

- validateStructure (62) — exactly one initial, ≥1 final, no orphans/dangling/duplicates/unreachable/no-path-to-final/final-with-outgoing, role-policy warnings.
- validateLiveData (269) — occupancy check so publishing never strands in-flight requests; honors statusRemap.
- validateGraph (363) — combines structural + status-definition + live data.

Graph editing (workflowGraph.service.ts)

upsertNodes/upsertEdges/deleteNodes/deleteEdges/replaceGraph — **every entry asserts the version is DRAFT** (lines 37–41), so a published graph can never be mutated in place.

Other engine services

- requestStatusDefinition.service.ts — status catalog CRUD/validation; normalizeStatusCode (uppercase [A-Z][A-Z0-9_]{1,99}), assertSelectableStatusCode, getStatusDefinitionForRuntime, getStatusDefinitionUsage (ref-counting).
- conditionalRules.service.ts — validates form showWhen conditional-field rules (validateConditionalRules, 144).
- conditionEvaluator.service.ts — typed AST evaluator for approval policies (EQ...NOT_IN, AND/OR/NOT), fail-closed, no arbitrary JS.

- `workflowBootstrap.service.ts` — **fail-closed planner (never writes)** that derives a candidate authoring graph from legacy steps + canonical definitions + global runtime policy rows + live occupancy; issues BLOCKING/WARNING BootstrapIssues.
- `workflowGraph.types.ts` — shared pure types (GraphNode, GraphEdge, WorkflowGraph, Finding, ValidationResult, RemapPlan).

10. Key Prisma models

Model	Line	Purpose / key FKs
<code>`Request`</code>	11 99	Core ticket. FK: <code>requestTypeId, serviceDeskId, requesterId/assignedToId→User</code> . SLA fields; timestamps <code>`resolvedAt/closedAt/completedAt`</code> ; <code>`formConfigSnapshot/Version`</code> ; <code>**`status` String` (catalog-backed)**</code> ; <code>`version` Int` optimistic concurrency</code> ; <code>`deletedAt`</code> ; <code>`@@unique([id,tenantId,departmentId])`</code>
<code>`RequestActivity`</code>	13 35	Timeline/comment; <code>`ActivityType` enum (COMMENT/STATUS_CHANGE/ASSIGNMENT/ATTACHMENT/SYSTEM/APPROVAL/REJECTION)</code> ; <code>`isSystemGenerated`</code> , <code>`isInternal`</code>
<code>`RequestAttachment`</code>	13 65	Uploaded file; <code>`AttachmentScanStatus/Classification/RetentionStatus`</code>
<code>`RequestParticipant`</code>	33 92	Shared-with; <code>`participantRole`</code> ; <code>`@@unique([requestId,userId,participantRole])`</code>
<code>`RequestApproval`</code>	15 42	Approval record; <code>`approverType`</code> , <code>`approverId`</code> , <code>`entityId`</code> , <code>`policyId`</code> , delegation, <code>`dueAt`</code> , <code>`status`</code>
<code>`RequestType`</code>	47 6	Catalog item; FK <code>`workflowTypeId→WorkflowType`</code> ; <code>`slaHours`</code> , <code>`formConfig` Json</code> , <code>`classification`</code> , <code>`lifecycleStatus`</code>
<code>`WorkflowType`</code>	86 5	Workflow container; <code>`code` unique</code> ; steps/versions/requestTypes
<code>`WorkflowStep`</code>	88 3	Compiled step (runtime); <code>`slaPause`</code> ; <code>`@@unique([workflowTypeId,status])`</code>
<code>`WorkflowVersion`</code>	92 0	Authoring version; <code>`WorkflowVersionStatus` DRAFT/ACTIVE/ARCHIVED</code> ; <code>`@@unique([workflowTypeId,version])`</code>
<code>`WorkflowNode`</code>	94 1	STATUS node; FK <code>`statusCode→RequestStatusDefinition.code`</code> ; <code>`isInitial/isFinal/slaPause/icon/config`</code>
<code>`WorkflowEdge`</code>	97 2	Directed transition; <code>`transitionLabel`</code> , <code>`requiresComment`</code> , auto-assign, allowedRoles; <code>`@@unique([workflowVersionId,fromNodeId,toNodeId])`</code>
<code>`WorkflowTransition`</code>	22 93	<code>**Runtime source of truth.** scope <code>`tenantId`+`workflowTypeId` (NULL=all, most-specific-first)</code>; <code>`@@unique([tenantId,workflowTypeId,fromStatus,toStatus])`</code></code>
<code>`WorkflowHistory`</code>	99 8	Immutable transition history (P04-15); <code>`requestVersion`</code>
<code>`WorkflowCommandResult`</code>	10 26	Idempotency ledger; <code>`@@unique([tenantId,idempotencyKey])`</code>
<code>`RequestStatusDefinition`</code>	22 68	Status <code>**catalog**</code> ; <code>`code` unique</code> , <code>`label`</code> , <code>`category`</code> , <code>`lifecycleType` (OPEN/RESOLVED/CLOSED/CANCELLED)</code> , <code>`isActive`</code> , <code>`retiredAt`</code>
<code>`Entity`</code>	23 32	Approver entity for chargeback routing (<code>`RequestTypeEntityRouting`</code>)

11. Quirks / gotchas for the maintainer

- WorkflowFormSchema / WorkflowFormResponse **do not exist** as models — forms are Json on RequestType/Request.
- Request.status **is a catalog-backed** String, not the RequestStatus enum, with a legacy compat shim (requestStatusCompat.ts keeps LEGACY_REQUEST_STATUS_CODES).
- updateStatus **in the controller still performs some side effects** (auto-approval creation, auto-assign, notifications) *outside* the transactional workflowCommand boundary — a potential partial-failure surface.
- **Initial status is hardcoded per request-type code** in createRequest (not purely workflow-driven).